Smart Seller OS

Production-Grade SaaS Master Product Requirements Document

Document Version: 1.0
Status: Product Baseline / Production Specification
Product Type: Multi-Tenant B2B SaaS
Target Users: E-commerce & F-commerce Business Owners
Architecture: Modular Monolith
Development Approach: Specification-Driven, API-First
Target: Production-Grade / Commercial SaaS

---

1. Product Vision

1.1 Overview

Smart Seller OS is a cloud-based, multi-tenant SaaS platform designed specifically for **e-commerce and F-commerce sellers**.

The platform brings the most important daily seller operations into one centralized system:

- Order management
- Customer management
- Customer conversations
- Order confirmation
- Fake/suspicious order detection
- Delivery and return tracking
- Product and inventory management
- Seller analytics
- Automation
- Notifications
- Business reporting
- AI-powered operational assistance

The goal is not to build another basic order management system.

The long-term objective is to build a complete:

> **E-commerce Business Operating System for Online Sellers**

The platform should allow a seller to manage customers, orders, communication, inventory, delivery operations, fraud risk, analytics and automation from a single workspace.

---

2. Problem Statement

Online sellers frequently manage their business using multiple disconnected tools.

Typical workflows may involve:

- Facebook Messenger
- Facebook Pages
- WhatsApp
- Google Sheets
- Excel
- Courier dashboards
- Payment systems
- Manual phone calls
- Separate order management software
- Manual customer verification

This creates several problems:

Operational Problems

- Orders are scattered across platforms.
- Customer conversations are difficult to track.
- Order confirmation is slow.
- Employees duplicate work.
- Order status is often outdated.
- Sellers lack a centralized customer history.

Financial Problems

- Fake orders create delivery costs.
- Returned orders reduce profit.
- COD failures create operational losses.
- Suspicious customers may repeatedly place orders.
- Sellers have limited visibility into actual business performance.

Scalability Problems

As order volume increases, manual processes become difficult to maintain.

Therefore, Smart Seller OS should provide a unified operational platform.

---

3. Product Goals

The system must:

1. Centralize seller operations.
2. Provide complete order lifecycle management.
3. Centralize customer conversations.
4. Track customer history.
5. Detect potentially suspicious/fake orders.
6. Reduce failed delivery and return losses.
7. Manage products and inventory.
8. Support multiple sales channels.
9. Provide business analytics.
10. Automate repetitive operational tasks.
11. Support multiple organizations/tenants.
12. Provide secure role-based access.

13. Support SaaS subscriptions.
14. Provide API and webhook integrations.
15. Maintain complete auditability.
16. Be scalable without premature microservices.
17. Support future AI-powered business features.

---

4. Target Customers

Primary Customers

Small Online Sellers

- Facebook sellers
- Instagram sellers
- Home-based businesses
- Small e-commerce stores

Growing Businesses

- Multi-product sellers
- Multi-employee seller teams
- Multi-channel sellers
- High-volume COD businesses

Established Businesses

- E-commerce companies
- Retail businesses
- D2C brands
- Multi-store businesses
- Agencies managing multiple seller accounts

---

5. Core Product Modules

The platform will contain the following major domains:

1. Authentication & Identity
2. Organization / Tenant Management
3. User Management
4. Roles & Permissions
5. Customer Management / CRM
6. Product Management
7. Inventory Management
8. Order Management
9. Order Pipeline
10. Customer Messaging
11. Order Confirmation
12. Fake Order Detection
13. Courier / Delivery Management

14. Returns & Failed Deliveries
15. Payments & COD
16. Notifications
17. Automation
18. Analytics & Reporting
19. Global Search
20. Audit Logging
21. API Platform
22. Webhooks
23. SaaS Subscription & Billing
24. Platform Administration
25. Feature Flags
26. AI / Intelligence Layer
27. System Observability
28. Security & Compliance

---

6. Multi-Tenant Architecture

Smart Seller OS must be a true multi-tenant SaaS.

Each business is an independent tenant.

Example:

```
Smart Seller OS
│
├── Tenant A
│   ├── Users
│   ├── Customers
│   ├── Orders
│   ├── Products
│   └── Inventory
│
├── Tenant B
│   ├── Users
│   ├── Customers
│   ├── Orders
│   ├── Products
│   └── Inventory
│
└── Tenant C
```

Tenant Isolation

A tenant must never be able to:

- Read another tenant's data
- Update another tenant's data
- Delete another tenant's data
- Search another tenant's data
- Infer protected information from another tenant

Tenant isolation must be enforced server-side.

Frontend filtering must never be considered a security boundary.

Conceptual request flow:

Request
↓
Authentication
↓
Tenant Resolution
↓
Authorization
↓
Validation
↓
Business Logic
↓
Repository
↓
Database

---

7. Organization Management

Each seller organization should have:

- Organization name
- Logo
- Business type
- Industry
- Contact information
- Address
- Country
- Currency
- Timezone
- Language
- Tax configuration
- Business settings
- Order settings
- Notification settings
- Fraud detection settings

Organization status:

TRIAL
ACTIVE
SUSPENDED
CANCELLED
DELETED

---

8. Authentication & Identity

The system must support:

- Registration
- Login
- Logout
- Email verification
- Password reset
- Password change
- Session management
- Session revocation
- Failed login protection
- Account status validation

Future authentication:

- Google OAuth
- Microsoft OAuth
- OpenID Connect
- SAML SSO
- Passkeys / WebAuthn

---

9. User Management

Tenant administrators can:

- Invite users
- Resend invitations
- Activate users
- Suspend users
- Deactivate users
- Assign roles
- Remove roles
- Force logout
- View active sessions

User lifecycle:

INVITED
↓
ACTIVE
↓
SUSPENDED
↓
DEACTIVATED

---

10. RBAC & Permissions

The system must support:

- System roles
- Custom roles
- Permission-based access
- Resource ownership checks
- Tenant-level restrictions

Example permissions:

orders.create
orders.read
orders.update
orders.delete
orders.confirm
orders.cancel
orders.refund

customers.create
customers.read
customers.update
customers.delete

messages.read
messages.send

fraud.read
fraud.review
fraud.override

products.create
products.read
products.update
products.delete

inventory.read
inventory.update

reports.read

Example roles:

OWNER
ADMIN
MANAGER
ORDER_MANAGER
CUSTOMER_SUPPORT
INVENTORY_MANAGER
SALES_AGENT
ACCOUNTANT
VIEWER

---

11. Customer Management / CRM

The customer module is one of the core domains.

Each customer may contain:

- Name
- Phone
- Alternative phone
- Email
- Address

- City
- Area
- Postal code
- Notes
- Tags
- Customer status
- Total orders
- Successful orders
- Cancelled orders
- Returned orders
- Total spending
- Last order date
- Risk score
- Communication history

---

12. Customer Profile

A customer profile should provide a complete 360° view.

Example:

```
Customer
│
├── Basic Information
├── Contact Information
├── Addresses
├── Order History
├── Payment History
├── Delivery History
├── Return History
├── Conversation History
├── Fraud/Risk History
├── Notes
├── Tags
└── Activity Timeline
```

The seller should be able to understand a customer's history without checking multiple systems.

---

13. Order Management

Order management is the primary MVP domain.

The platform must support the complete order lifecycle.

```
ORDER RECEIVED
    ↓
PENDING REVIEW
    ↓
CONFIRMATION
    ↓
CONFIRMED
```

↓
PROCESSING
↓
READY TO SHIP
↓
SHIPPED
↓
OUT FOR DELIVERY
↓
DELIVERED

Alternative states:

CANCELLED
RETURNED
FAILED_DELIVERY
ON_HOLD
SUSPICIOUS

---

14. Order Creation

Orders may originate from:

- Manual seller entry
- Website
- Facebook
- Instagram
- WhatsApp
- API
- External e-commerce platform
- CSV import
- Future marketplace integrations

Each order should contain:

- Order ID
- Customer
- Products
- Quantity
- Price
- Discount
- Shipping fee
- Tax
- Total
- Payment method
- Payment status
- Delivery address
- Courier
- Tracking number
- Order source
- Order status
- Fraud/risk status
- Notes
- Created by
- Created date

- Updated date

---

15. Order Pipeline

The seller should have a Kanban-style order pipeline.

Example:

New
 │
 ├── Pending Confirmation
 │
 ├── Confirmed
 │
 ├── Processing
 │
 ├── Ready to Ship
 │
 ├── Shipped
 │
 ├── Delivered
 │
 ├── Returned
 │
 └── Cancelled

Users with appropriate permissions can move orders between stages.

Important state transitions must be validated server-side.

---

16. Order Confirmation

Order confirmation is a core operational workflow.

The system should support:

- Manual confirmation
- Phone confirmation
- SMS confirmation
- Messaging confirmation
- Automated confirmation
- Confirmation notes
- Confirmation attempts
- Confirmation timestamps
- Confirmation agent

Example:

New Order
  ↓
Risk Check

↓
Customer Verification
↓
Confirmation
↓
Confirmed / Rejected

---

17. Fake Order Detection

Fake order detection is a core differentiating feature.

The system should analyze order and customer-related signals to calculate a risk score.

Example signals:

- Previous order history
- Previous successful deliveries
- Previous returns
- Failed deliveries
- Cancellation history
- Multiple orders in short periods
- Repeated phone number
- Suspicious address patterns
- Order frequency
- COD behavior
- Customer risk history
- Device/IP information where legally and technically appropriate

---

18. Fraud / Risk Score

Each order may have:

Risk Score: 0 - 100

Example:

0–20 LOW RISK
21–50 MEDIUM RISK
51–75 HIGH RISK
76–100 CRITICAL RISK

The system should not automatically claim that an order is fraudulent merely because the score is high.

Instead:

Risk Detection
↓
Risk Score
↓
Risk Factors
↓
Seller Review

↓
Approve / Hold / Reject

---

19. Fraud Detection Explainability

The system should show why an order received a risk score.

Example:

Risk Score: 82

Reasons:
✓ 4 previous failed deliveries
✓ 3 recent cancelled orders
✓ New customer
✓ High-value COD order
✓ Multiple orders from same phone

This makes the system understandable and actionable for sellers.

---

20. Fraud Review

Authorized users can:

- View suspicious orders
- Review risk factors
- Approve order
- Reject order
- Put order on hold
- Mark false positive
- Add review notes

Every manual fraud decision should be auditable.

---

21. Customer Messaging

The platform must provide centralized customer communication.

Potential channels:

- Facebook Messenger
- WhatsApp
- SMS
- Email
- In-app messaging

MVP should initially focus on the highest-value supported channel integrations.

---

22. Unified Inbox

The seller should have one centralized inbox.

Example:

Unified Inbox
│
├── New
├── Waiting for Reply
├── Awaiting Confirmation
├── Customer Replied
├── Resolved
└── Assigned to Me

Each conversation should support:

- Customer identity
- Channel
- Messages
- Timestamp
- Assigned agent
- Conversation status
- Internal notes
- Tags
- Order context

---

23. Order + Conversation Context

The system should connect conversations with orders.

Example:

Customer: Rahim

Conversation
↓
Order #ORD-10231
↓
Product: T-Shirt
↓
Amount: ৳1,250
↓
Status: Pending Confirmation

The seller should not need to search for the order separately.

---

24. Messaging Automation

Future automation should support:

New Order
↓
Send Confirmation Message
Order Shipped
↓
Send Tracking Message
Delivery Failed
↓
Notify Customer Support
Customer Doesn't Reply
↓
Create Follow-up Task

---

25. Product Management

Products should support:

- Product name
- SKU
- Barcode
- Category
- Brand
- Description
- Cost price
- Selling price
- Discount
- Tax
- Status
- Images
- Variants

Variants may include:

- Size
- Color
- Model
- Packaging

---

26. Inventory Management

Inventory should support:

- Current stock
- Reserved stock
- Available stock
- Minimum stock
- Stock adjustment
- Stock transfer
- Stock reservation
- Low-stock alerts
- Out-of-stock alerts

Stock movement types:

PURCHASE
SALE
TRANSFER
RETURN
ADJUSTMENT
DAMAGE

Every stock movement must be traceable.

---

27. Courier / Delivery Management

The platform should provide a courier abstraction layer.

Courier information:

- Courier name
- Tracking ID
- Delivery status
- Shipment date
- Delivery date
- Delivery fee
- COD amount
- Return status

Future integrations may include multiple local and international courier providers.

The core order domain should not be tightly coupled to one courier provider.

---

28. Returns & Failed Delivery

The system must track:

- Return request
- Return reason
- Failed delivery
- Customer refusal
- Wrong address
- Courier failure
- Product damage
- Refund
- Replacement

Return lifecycle:

RETURN REQUESTED
↓
APPROVED
↓
PICKUP

↓
RECEIVED
↓
INSPECTED
↓
REFUNDED / REPLACED

---

29. Payment Management

Support:

- Cash on Delivery
- Bank transfer
- Card
- Online payment
- Manual payment

Payment statuses:

PENDING
PARTIALLY_PAID
PAID
FAILED
REFUNDED

Payment providers must use an abstraction layer.

---

30. Seller Analytics

The dashboard should provide actionable business metrics.

Core metrics:

- Total orders
- New orders
- Confirmed orders
- Delivered orders
- Cancelled orders
- Returned orders
- Failed deliveries
- Revenue
- Profit estimate
- Average order value
- Delivery success rate
- Return rate
- Cancellation rate
- Customer growth
- Repeat customer rate
- Fake-order risk rate

---

31. Fraud Analytics

Dedicated fraud dashboard:

Total Orders
↓
Risk Distribution
↓
Low / Medium / High / Critical

Metrics:

- Suspicious orders
- Rejected orders
- False positives
- Failed delivery rate
- Return rate
- Customer risk distribution
- Loss prevented estimate

---

32. Reporting

Reports should include:

- Sales report
- Order report
- Customer report
- Product report
- Inventory report
- Delivery report
- Return report
- Fraud report
- Revenue report

Exports:

- CSV
- Excel
- PDF

Large exports must run asynchronously.

---

33. Notifications

Supported channels:

IN_APP
EMAIL
SMS
PUSH

WEBHOOK

Notification system should support:

- Templates
- User preferences
- Read/unread state
- Delivery status
- Retry
- Notification history

---

34. Workflow Automation

The platform should support configurable automation.

Core model:

Trigger
↓
Condition
↓
Action

Examples:

Order Created
↓
Risk Score > 70
↓
Mark Order For Review
Order Confirmed
↓
Send Customer Message
Order Delivered
↓
Send Review Request
Low Stock
↓
Notify Inventory Manager

Automation should run asynchronously when required.

---

35. AI / Intelligence Layer

AI should be introduced after the core operational system is stable.

Potential AI features:

AI Order Risk Analysis

Analyze order/customer signals and generate risk explanations.

AI Customer Assistant

Assist customer support agents with:

- Reply suggestions
- Conversation summaries
- Customer intent detection
- Order-related answers

AI Business Assistant

Example:

> "Why did my revenue decrease this week?"

The system can analyze available business data and provide an explanation.

AI Insights

Potential insights:

- High-return products
- High-risk customers
- Low-performing products
- Delivery problems
- Customer retention opportunities
- Inventory risks

AI must not bypass authorization or expose cross-tenant information.

---

36. Global Search

Search should support:

- Customers
- Orders
- Products
- Conversations
- Tracking IDs
- Phone numbers
- Invoices
- Users

Support:

- Search
- Filtering
- Sorting
- Pagination

Search infrastructure should be replaceable with dedicated search technology if scale requires it.

---

37. Audit Logging

Important business and security events must be audited.

Each event should capture:

Who
What
When
Where
Resource
Resource ID
Before
After
Tenant
User
Request ID
Correlation ID

Example:

User: Admin

Action: UPDATE

Resource: Order
Resource ID: ORD-10231

Before:
status = PENDING

After:
status = CONFIRMED

Audit logs must not be modifiable by normal tenant users.

---

38. API Platform

Backend must be API-first.

Requirements:

- REST
- JSON
- API versioning
- Authentication
- Authorization
- Validation
- Pagination
- Filtering
- Sorting
- Rate limiting
- Consistent errors

- OpenAPI documentation

Example:

/api/v1/orders
/api/v1/customers
/api/v1/products
/api/v1/inventory
/api/v1/conversations
/api/v1/fraud
/api/v1/reports

---

39. Webhooks

The system should support outbound webhooks.

Events:

order.created
order.confirmed
order.shipped
order.delivered
order.cancelled
order.returned

customer.created

payment.created
payment.completed

product.created

conversation.created

Webhook system must support:

- Signing
- Retry
- Delivery logs
- Failure handling
- Replay
- Event filtering

Webhook delivery should be asynchronous.

---

40. API Keys

Organizations may create API keys.

Support:

- Key creation

- Revocation
- Rotation
- Expiration
- Permission scopes
- Last-used timestamp

Secret material must never be stored in plaintext.

---

41. SaaS Subscription & Billing

Smart Seller OS itself must operate as a subscription SaaS.

Initial plans:

FREE
STARTER
BUSINESS
PROFESSIONAL
ENTERPRISE

Plans may control:

- Users
- Orders/month
- Customers
- Products
- Storage
- Messaging channels
- Automation
- API requests
- Reports
- AI usage

---

42. Usage-Based Entitlements

Example:

STARTER

Users: 3
Orders: 1,000/month
Products: 500
Automation: 10 workflows
API: 10,000 requests

The entitlement system should be configurable rather than hardcoded.

---

43. Subscription Lifecycle

Support:

TRIAL
ACTIVE
PAST_DUE
GRACE_PERIOD
CANCELLED
EXPIRED

Operations:

- Trial
- Subscribe
- Upgrade
- Downgrade
- Renewal
- Cancellation
- Grace period
- Billing invoice
- Coupon
- Discount

---

44. Platform Administration

The SaaS provider requires a separate platform admin panel.

Platform admins can manage:

- Organizations
- Users
- Subscriptions
- Plans
- Revenue
- Usage
- System health
- Security events
- Audit logs
- Feature flags
- Support operations

Platform-level permissions must be separated from tenant-level permissions.

---

45. Feature Flags

Feature flags should support controlled releases.

Examples:

ai_order_risk = true

whatsapp_integration = false

advanced_analytics = true

automation_engine = false

Flags may target:

- Environment
- Tenant
- Subscription plan
- User cohort

---

46. Security Requirements

The system must implement:

- Strong password hashing
- Secure session/token management
- Input validation
- Rate limiting
- Brute-force protection
- Secure CORS
- CSRF protection where applicable
- XSS protection
- Security headers
- Parameterized database queries
- Secret management
- TLS
- Encryption at rest where appropriate
- Least-privilege authorization
- Secure file validation
- Dependency vulnerability scanning

Security-sensitive operations must be audited.

---

47. Data Privacy

The platform should support:

- Data export
- Account deletion requests
- Data retention
- Privacy settings
- Consent management where applicable

Privacy requirements should be configurable based on target markets.

---

48. Performance Requirements

Initial production targets:

API

Normal CRUD API:

p95 < 300ms

under expected baseline load.

Requirements:

- Database indexes
- Connection pooling
- Pagination
- Efficient queries
- Redis where justified
- Async processing
- Rate limiting

Long-running operations must not block normal API requests.

---

49. Background Jobs

Background workers should handle:

- Email
- SMS
- Notifications
- Message processing
- Report generation
- PDF generation
- CSV/Excel exports
- Webhooks
- Webhook retries
- Automation
- Scheduled tasks
- AI processing where appropriate

Architecture:

API
↓
Queue
↓
Worker
↓
Job

Jobs must support:

- Retry

- Idempotency
- Failure handling
- Logging
- Monitoring

---

50. Observability

Structured logs must include:

timestamp
level
environment
service
request_id
correlation_id
tenant_id
user_id
message
error

Never log:

- Passwords
- Access tokens
- API secrets
- Payment secrets
- Sensitive customer data unnecessarily

Metrics:

- Request count
- Error rate
- Latency
- Database latency
- Queue depth
- Worker status
- Authentication failures
- Job failures
- Resource usage

The system should be OpenTelemetry-ready.

---

51. Health Checks

Operational endpoints:

/health
/live
/ready

Health checks should monitor:

- Application
- Database
- Redis
- Queue
- Required external providers

Sensitive information must never be exposed.

---

52. Backup & Disaster Recovery

Requirements:

- Automated database backup
- Retention policy
- Restore testing
- Point-in-time recovery where available
- Object storage backup
- Disaster recovery documentation

Backups must periodically be restored and verified.

---

53. Database Requirements

Primary database:

PostgreSQL

Requirements:

- Version-controlled migrations
- Foreign keys
- Constraints
- Indexes
- Transactions
- Connection pooling
- Unique constraints
- Audit timestamps
- Soft deletion where appropriate
- Optimistic locking where necessary

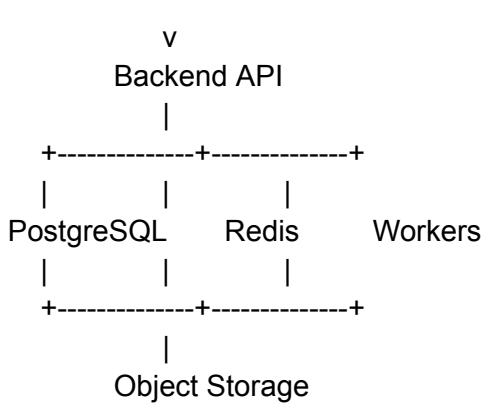
Every tenant-owned table should have a clear tenant relationship.

---

54. Architecture

Initial production architecture:

```
Next.js
  |
```



Architecture style:

Modular Monolith

The application should not begin with microservices.

---

55. Backend Domain Modules

Recommended modules:

auth
organization
user
role
customer
product
inventory
order
order-pipeline
conversation
fraud
courier
delivery
return
payment
notification
automation
analytics
reporting
subscription
billing
audit
api
webhook
platform-admin

Each module should maintain clear domain boundaries.

---

56. Internal Module Structure

Each module should follow a consistent structure:

```
module/
├── domain/
├── application/
├── repository/
├── handler/
├── dto/
├── mapper/
├── validation/
└── tests/
```

Business logic must not be placed directly inside HTTP handlers.

---

57. Scalability Strategy

Stage 1 — MVP

Modular Monolith
PostgreSQL
Redis
Background Workers
Object Storage

Stage 2 — Growth

Horizontal API Scaling
Dedicated Workers
CDN
Read Replicas
Dedicated Search

Stage 3 — Large Scale

Only proven high-load domains should become separate services.

Potential candidates:

Messaging
Search
Analytics
Workflow
AI
Notifications
File Processing

Microservices should be introduced based on measurable requirements.

---

58. Testing Strategy

Backend

- Unit tests
- Integration tests
- Repository tests
- API tests
- Authorization tests
- Tenant isolation tests
- Fraud detection tests
- Security tests

Frontend

- Component tests
- Integration tests
- E2E tests

System

- Load tests
- Stress tests
- Migration tests
- Failure recovery tests

Critical business rules must have automated coverage.

---

59. CI/CD

Recommended pipeline:

Push
↓
Lint
↓
Format Check
↓
Type Check
↓
Unit Tests
↓
Integration Tests
↓
Security Scan
↓
Build
↓
Docker Build
↓
Deploy
↓
Smoke Tests

Production deployment should support:

- Controlled release
- Rollback
- Environment configuration
- Migration safety

---

60. Frontend Requirements

Recommended:

Next.js
React
TypeScript

Requirements:

- Responsive UI
- Accessibility
- Loading states
- Empty states
- Error states
- Permission-aware UI
- Form validation
- Error boundaries
- Secure authentication
- Internationalization-ready UI

Frontend authorization is only for UX.

Backend authorization remains authoritative.

---

61. Main Seller Dashboard

Dashboard should show:

Today's Orders
Today's Revenue
Pending Confirmation
High Risk Orders
Pending Shipments
Failed Deliveries
Returns
Low Stock
Unread Messages

Analytics:

Revenue Trend
Order Trend
Delivery Success

Return Rate
Cancellation Rate
Customer Growth
Fraud Risk

---

62. MVP Scope

The first commercial MVP should NOT attempt to build the entire platform.

The MVP should focus on the actual seller pain points.

MVP Phase 1

1. Authentication

- Registration
- Login
- Email verification
- Password reset

2. Organization

- Create organization
- Organization settings
- Tenant isolation

3. User & RBAC

- Invite users
- Roles
- Permissions

4. Customer

- Customer CRUD
- Customer profile
- Customer history

5. Product

- Product CRUD
- Categories
- Variants

6. Order Management

- Create order
- Order pipeline
- Order status
- Order confirmation
- Order search/filter

7. Fake Order Detection

- Risk score
- Risk factors
- Suspicious order queue
- Manual review
- Approve/reject

8. Customer Messaging

Initially support the highest-priority communication channel.

- Inbox
- Conversations
- Customer context
- Order context
- Internal notes

9. Delivery

- Courier assignment
- Tracking
- Delivery status
- Failed delivery
- Return tracking

10. Inventory

- Stock
- Stock adjustment
- Stock reservation
- Low-stock alert

11. Notifications

- In-app
- Email

12. Dashboard

- Orders
- Revenue
- Delivery
- Returns
- Fraud metrics

13. Audit Logs

Important business and security actions.

14. API

Versioned REST API.

15. SaaS Billing

- Plans
- Trial
- Subscription

- Usage limits

16. Platform Admin

- Tenant management
- User management
- Subscription management
- System monitoring

---

63. Post-MVP Features

After MVP stability:

Phase 2

- Multiple courier integrations
- WhatsApp integration
- Facebook integration
- Advanced messaging
- Automation engine
- Advanced analytics
- Bulk operations
- Advanced inventory
- Customer segmentation

Phase 3

- AI customer assistant
- AI business assistant
- AI order risk model
- AI analytics
- Recommendation engine
- Predictive delivery risk

Phase 4

- Enterprise SSO
- SAML
- SCIM
- White labeling
- Custom domains
- Advanced RBAC
- Data residency
- Enterprise SLA
- Dedicated infrastructure

---

64. Definition of Done

A feature is not complete until applicable requirements are satisfied.

Minimum criteria:

- Requirement implemented
- Database migration completed
- Validation implemented
- Authorization implemented
- Tenant isolation verified
- Error handling implemented
- Unit tests added
- Integration tests added where appropriate
- API documentation updated
- Audit requirements implemented
- Logging implemented
- Security reviewed
- Edge cases handled
- Code reviewed
- Documentation updated

---

65. Development Workflow

Every major feature must follow:

Business Requirement
↓
Product Specification
↓
Domain Design
↓
Database Design
↓
API Contract
↓
Implementation
↓
Tests
↓
Security Review
↓
Code Review
↓
Acceptance
↓
Deployment

AI-assisted development must not bypass this process.

AI tools must follow approved specifications and existing architecture.

---

66. Recommended Documentation Structure

docs/

```
│
├── requirements/
│   ├── 00-master-requirements.md
│   ├── 01-authentication.md
│   ├── 02-multi-tenancy.md
│   ├── 03-organization.md
│   ├── 04-user-management.md
│   ├── 05-rbac.md
│   ├── 06-customers.md
│   ├── 07-products.md
│   ├── 08-inventory.md
│   ├── 09-orders.md
│   ├── 10-order-pipeline.md
│   ├── 11-messaging.md
│   ├── 12-fraud-detection.md
│   ├── 13-delivery.md
│   ├── 14-returns.md
│   ├── 15-payments.md
│   ├── 16-notifications.md
│   ├── 17-automation.md
│   ├── 18-reporting.md
│   ├── 19-subscription.md
│   ├── 20-audit.md
│   ├── 21-api.md
│   ├── 22-webhooks.md
│   ├── 23-platform-admin.md
│   ├── 24-security.md
│   └── 25-ai.md
│
├── architecture/
│   ├── system-architecture.md
│   ├── database-architecture.md
│   ├── api-architecture.md
│   └── background-jobs.md
│
└── decisions/
    └── ADRs
```

---

67. Product Success Metrics

The platform should ultimately measure:

Operational Efficiency

- Order processing time
- Confirmation time
- Messages handled per agent
- Automation rate

Business Performance

- Revenue
- Average order value

- Repeat customer rate
- Delivery success rate

Loss Prevention

- Return rate
- Failed delivery rate
- Suspicious order rate
- Fraud loss prevented

Product Adoption

- Active tenants
- Active users
- Orders processed
- Messages processed
- Automation usage
- AI feature usage

SaaS Metrics

- MRR
- ARR
- Churn
- Conversion rate
- Trial-to-paid rate
- Customer lifetime value

---

68. Long-Term Product Vision

Smart Seller OS should evolve from:

Order Management

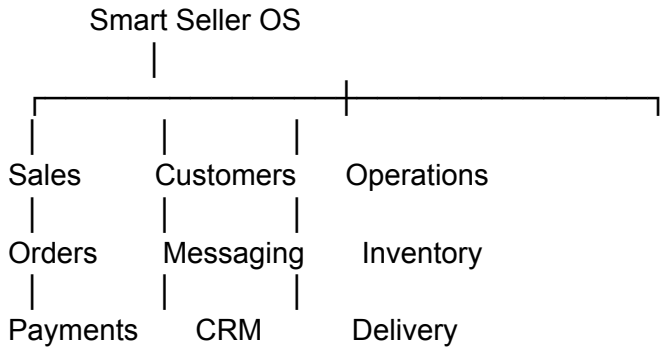
into:

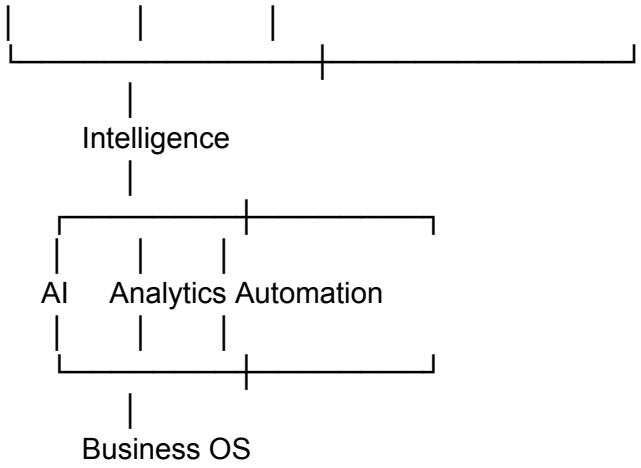
Seller Operations Platform

and ultimately:

E-commerce Business Operating System

Long-term platform:





The final product should not become a collection of unrelated CRUD modules.

Every module should contribute to the central objective:

> **Help online sellers operate their business faster, reduce avoidable losses, understand their customers, automate repetitive work, and make better business decisions.**

---

69. Final Product Direction

The product's strategic priorities are:

1. Seller-focused product design
2. Strong multi-tenancy
3. Security by design
4. Role and permission-based authorization
5. Centralized order management
6. Unified customer communication
7. Fake-order risk detection
8. Delivery and return visibility
9. Actionable analytics
10. Workflow automation
11. API-first architecture
12. Reliable background processing
13. Comprehensive auditability
14. Production observability
15. Commercial SaaS billing
16. AI-ready architecture
17. Horizontal scalability
18. Enterprise extensibility

The MVP should remain focused on:

> **Orders + Customers + Messaging + Fake Order Detection + Delivery + Inventory + Analytics**

while the architecture should be strong enough to evolve into a complete:

> **E-commerce Business Operating System**

without requiring a complete rewrite.

---

70. Core Product Principle

The most important product principle is:

> **Do not build features simply because they are possible. Build features that reduce operational workload, prevent business losses, or improve seller decision-making.**

Every future feature should answer at least one question:

Does this save the seller time?

Does this reduce operational cost?

Does this prevent financial loss?

Does this improve customer experience?

Does this improve business visibility?

Does this enable automation?

Does this enable better decisions?

If a feature does not meaningfully contribute to these goals, it should not become a core product priority.